

To train a software program or construct an algorithm to automatically calculate your **Environmental Risk Score (ERS)**, you need to build a logical data pipeline. Your handwritten notes outline a very clear, deterministic formula. Because this relies on strict mathematical logic rather than pattern recognition, you don't need a complex machine learning model—you need a **deterministic algorithm** backed by a structured database.

Here is the step-by-step architecture to build and program your software to handle this calculation.

1. The Core Logic & Architecture

The software needs to execute three primary functions: **Data Retrieval** (fetching P and B), **Dynamic Evaluation** (calculating the Potency Factor based on dates), and **Matrix Categorization** (scoring the outcome).

Your Formula Variables:

- **P (Persistence/Hazard Score):** Derived from the lowest PNEC value.
- **B (Bioaccumulation Score):** Derived from the Log K_{ow} value.
- **PF (Potency Factor):** A dynamic multiplier based on how long the drug has been expired.

2. Step-by-Step Software Workflow

Step A: Set Up the Database

Your software must have access to a reference database (like the database sheets we discussed from PubChem and the AMR Industry Alliance). You must assign static scoring weights to the values retrieved:

- **Persistence (P):** Code the software to lookup

the target API. If the PNEC is extremely low (highly toxic), assign a high persistence score (e.g., scale of 1 to 3 or 1 to 5).

- **Bioaccumulation (B):** Code the software to check the $\text{Log } K_{ow}$. If $\text{Log } K_{ow} < 3$, set $B = 0$ or 1 . If $\text{Log } K_{ow} \geq 3$, scale the B score higher.

Step B: Program the Dynamic Potency Factor (PF)

To calculate the PF, the user will input the **Expiry Date** of the disposal batch. The software will compare that date against the **Current Date** to find the elapsed time:

You can program this logic using conditional if-else statements:

- if **Elapsed Months is 0 to 6 months**: Set $PF = 0.90$ (Medium Resistance Risk)
- if **Elapsed Months is 6 to 18 months**: Set PF

= 0.60 (High Resistance Risk)

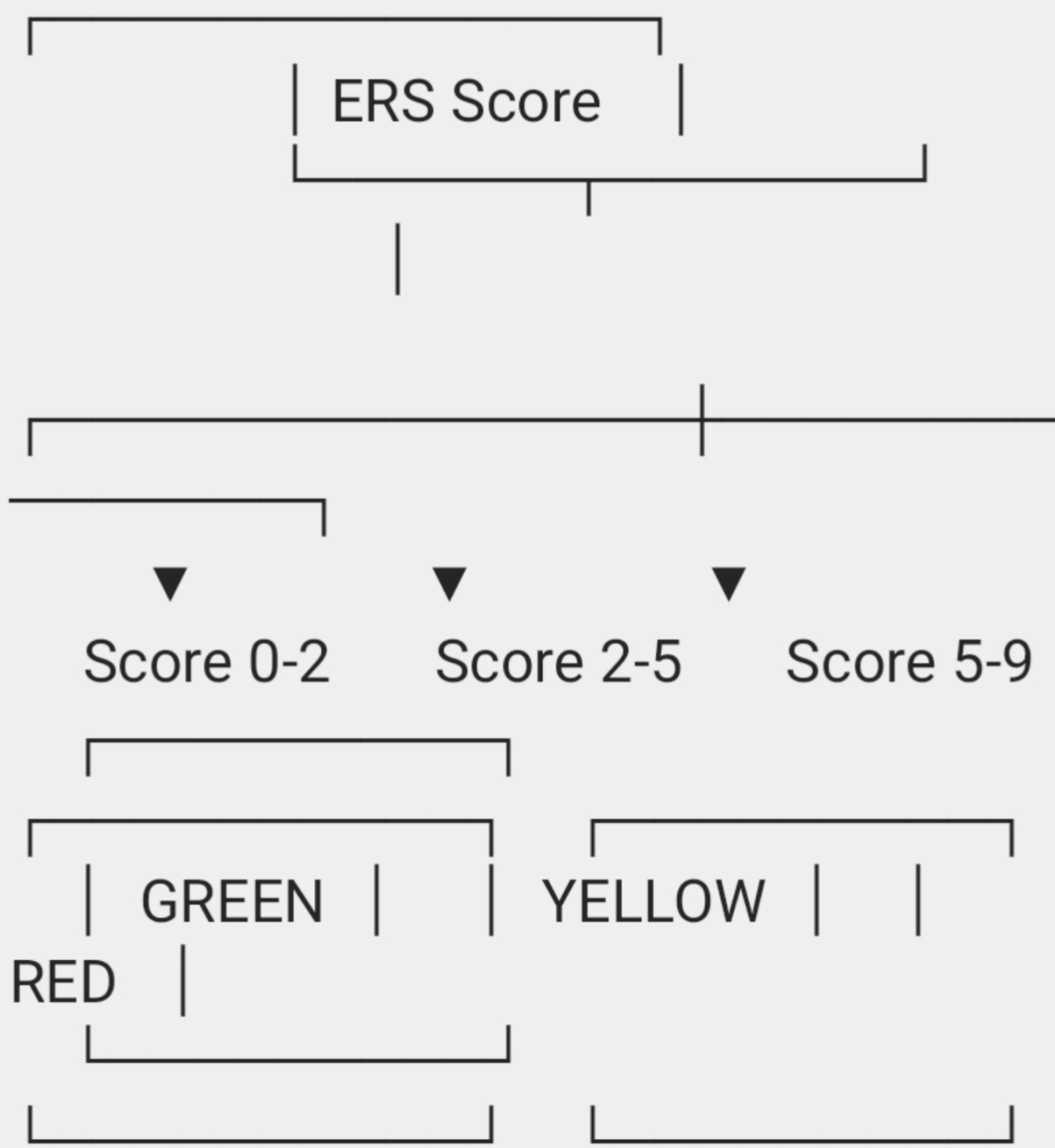
- if **Elapsed Months is > 18 months**: Set PF = 0.30 (Critical AMR Risk)

Note on Your Formula Logic: Your notes brilliantly highlight that sub-therapeutic (weak) levels of antibiotics drive superbugs. In your formula $\text{ERS} = (P + B) \times \text{PF}$, multiplying by 0.30 for older drugs actually *lowers* the final ERS score mathematically. If you want a drug older than 18 months to flag as **Critical/Highest AMR** and land in the **Red Zone (5-9)**, change the PF multiplier to work inversely (e.g., assigning a higher weight like PF = 2.0 or 3.0 for older, weaker degraded drugs), or switch the formula to a division step relative to concentration.

Step C: Execute the Calculation & Output Matrix

Once the variables are compiled, the code

computes the final score and checks it against your matrix boundaries to output a color-coded alert:



3. Writing the Blueprint Code (Python Example)

You can implement this logic directly into your platform's backend using this simple, clean Python structure:

python



```
from datetime import datetime

# 1. Mock Database representing your
# fetched API metrics
API_DATABASE = {
    "Amoxicillin": {"P_score": 1.5,
    "B_score": 0.5},
    "Ciprofloxacin": {"P_score": 2.0,
    "B_score": 0.2},
    "Azithromycin": {"P_score": 3.0,
    "B_score": 2.5}
}

def
```

```
calculate_potency_factor(expiry_date_str
):
```

```
    """Calculates months past expiry and
    assigns the PF value."""
```

```
    expiry_date =
    datetime.strptime(expiry_date_str, "%Y-
    %m-%d")
```

```
    current_date = datetime.now()
```

```
    # Calculate difference in total months
    elapsed_months = (current_date.year -
    expiry_date.year) * 12 +
    (current_date.month -
    expiry_date.month)
```

```
    if elapsed_months <= 6:
```

```
        return 0.90, "Medium Resistance"
```

```
    elif 6 < elapsed_months <= 18:
```

```
        return 0.60, "High Resistance"
```

```
    else:
```

```
        return 0.30, "Critical [Highest AMR]"
```

```
def
```

```
evaluate_environmental_risk(api_name,
    expiry_date_str):
```

```
# Fetch values from your reference
database
if api_name not in API_DATABASE:
    return "API not found in database
registry."
```

```
P = API_DATABASE[api_name]
["P_score"]
B = API_DATABASE[api_name]
["B_score"]
```

```
# Calculate dynamic potency factor
PF, risk_level =
calculate_potency_factor(expiry_date_str
)
```

```
# Execute your ERS Formula
ERS = (P + B) * PF
```

```
# Determine Matrix Categorization
if 0 <= ERS <= 2:
    classification = "Green -> Low
Environmental Impact"
elif 2 < ERS <= 5:
    classification = "Yellow -> Moderate
```



```
Potential Hazard"
```

```
else:
```

```
    classification = "Red -> Critical Risk!  
    Requires High-Temp Incineration."
```

```
    return {
```

```
        "Antibiotic": api_name,
```

```
        "ERS Score": round(ERS, 2),
```

```
        "Resistance Urgency": risk_level,
```

```
        "Disposal Protocol": classification
```

```
    }
```

```
# --- System Run Example ---
```

```
print(evaluate_environmental_risk("Azithr  
omycin", "2024-05-15"))
```

4. Turning This into a Smart Dashboard Application

To make this user-friendly for a pharmacy dashboard, hospital eco-management portal, or regulatory submission application, compile the

code behind an interface layout like this:

1. **Input Interface:** A simple web form containing a dropdown selection menu of the antibiotic names (linked to your API database) and a calendar date-picker for the batch expiry date.
2. **The Logic Pipeline:** When the user clicks "Calculate Risk Profile", the frontend passes those parameters directly into your backend code to match the values, execute the date-difference math, and run the matrix calculation.
3. **The Matrix Display:** The dashboard displays a clean interface output card styled directly to match your visual tracking framework (flashing a solid **Green**, **Yellow**, or **Red** card based on the computed value) alongside explicit regulatory disposal instructions (e.g., *"Requires High-Temperature Incineration"*).

